aRTsit PythonLib Documentation

Contents

Module `artistlib` 1

Getting started 2

About 2

Module `artistlib.junction` 28

Module `artistlib.material` 30

Module `artistlib.wrapper` 31

Module `artistlib.wrapper.artist_detector` 31

Module `artistlib.wrapper.artist_project` 33

Module `artistlib.wrapper.artist_send` 34

Module `artistlib.wrapper.artist_setup` 35

Module `artistlib.wrapper.artist_source` 40

Module `artistlib.wrapper.artistwrapper_all` 42

Module `artistlib.wrapper.material2` 48

Module `artistlib.wrapper.modules` 49

Module `artistlib.wrapper.part` 50

Module `artistlib.wrapper.start` 52

Module `artistlib.wrapper.start.artist_start` 52

Module `artistlib.wrapper.start.file_utility` 53

Module `artistlib.wrapper.start.find_installation` 54

Module `artistlib.wrapper.start.launcher` 54

Module `artistlib.wrapper.start.process_helper` 55

Module `artistlib.wrapper.start.ui` 55

Module `artistlib`

The aRTist Python library is designed to remotely control and automate the radiographic simulation software aRTist.

Hint: This project is in its early stages and may change significantly over time.

Getting started

Requirements

A **Python 3** environment (Python 3.8 or higher) is required. The following Python packages are required as well. They usually come with a Python environment, or can be installed there easily:

- none so far.

Installation

If you want to **install** the package in your Python environment, you can use pip[1]. For example, you can run the following command to make the toolbox available:

```
pip install artistlib
```

To use the package **without installation**, you need to download the package manually. You have the following three options:

- Download the package from PyPi[2]. You will get a zipped file called `artistlib-X.X.X.tar.gz` (where X.X.X is the current version number). Open or unpack this file to get to its contents.
- Download the repository from GitHub[3]: press the *clone* button to download a ZIP file. Unpack it or open it to see its contents.
- You can also clone the repository from GitHub via the terminal:

  ```
  git clone https://github.com/BAMresearch/aRTist-PythonLib.git
  ```

From any of these three options, you will get the complete package source code. It contains a folder called artistlib. If you want to use the library from your Python scripts without installing it, you need to copy the folder artistlib to the location of your Python scripts to make the package available.

About

The aRTist library was developed for utilize the Python scripting of the radiographic simulator aRTist[4].

The software is released under the **Apache 2.0 license,** its source code is available on GitHub[5].

Contributors

The following people contributed to code and documentation of the package:

- Alexander Funke
- David Schumacher
- Carsten Bellon
- David Plotzki
- David Denkler
- Julius Ott

Usage

Please make sure that aRTist is running and waiting for remote connection (menu: Tools>Enable remote access).

The following example code shows the basic usage:

[1] https://pip.pypa.io
[2] https://pypi.org/project/artistlib/
[3] https://github.com/BAMresearch/aRTist-PythonLib
[4] https://artist.bam.de
[5] https://github.com/BAMresearch/aRTist-PythonLib

```
import artistlib as a
# from artistlib.junction import Junction

# connect to aRTist (default: host='localhost', port=3658)
jct = a.Junction()


# print aRTist's version number
ver = jct.send("::aRTist::GetVersion full")
print(ver)

# - load a project
# - run the simulation
# - save the resulting projection from the image viewer
jct.send("FileIO::OpenAny $Xray(LibraryDir)/ExampleProjects/Schlumpfmark.aRTist")
jct.send("Engine::GoCmd")
ans = "Saved as: "
ans += jct.send(
    "Modules::Invoke ImageViewer SaveFile [file join $env(HOME) Pictures/artistlib.tif] true"
)
print(ans)

# - load a project
# - run the simulation without viewing the result
# - transfer the final projection
# - saving the projection image  >>>> This requires aRTist 2.12.7 or higher! <<<<
# - delete the images to release the memory
jct.send("FileIO::OpenAny $Xray(LibraryDir)/ExampleProjects/aRTist.aRTist")
jct.send("set imgList [Engine::Go]")
jct.send("RemoteControl::SendImage [lindex $imgList 0]")
jct.get_image("transferred.tif")
jct.send("foreach i $imgList {$i Delete}")
```

Usage of the wrapper

The following example code shows the basic usage of the Wrapper:

```
import artistlib as a
from pathlib import Path

project_folder = "$Xray(LibraryDir)/ExampleProjects/Dragon-CT.aRTist"
result_folder = Path.home() / "Pictures"
# Starts aRTist and activates remote control.
# a.ArtistAll() also starts aRTist and activates remote control.
a.start_artist()
wrapper = a.ArtistAll()
# Loads a project (.artist) or a part (.stl) based on the provided path.
wrapper.load_project(project_folder)

# Translation of the part with the given ID, to the given coordinates.
#
# part_orientation rotates the part with the given ID
# part_material sets the material of the part with the given ID.
wrapper.part_position_global(ID=1, x=10, y=10, z=-10)
wrapper.part_position_global(ID=1, x=30)
wrapper.part_orientation(1, angleX=90, angleY=45, angleZ=10)
wrapper.part_material(1, "Al")

# Translation of the detector.
```

```
# Rotation of the detector.
# detector_geo can change the Detector Geometry with Detector_Geo, which means changing either Pixel
wrapper.detector_position(x=30, y=20)
wrapper.detector_orientation(x=5)
wrapper.detector_geo(size=100, pixel=600)

# Translation of the source.
# Rotation of the source.
wrapper.source_position(z=155)
wrapper.source_orientation(x=1)

# Sets the individual values of the spectrum.
# Calculates the spectrum and saves it to a path.
wrapper.set_spectra(
    "Tube3", 300, "W", 12, 0.3, "Cu", 0.5, "Cu", 0.5, "Cu", 0.5, "Be", 1.0, 12, 1, 1.0
)
wrapper.cal_spectra(spec_name="Tube3_spc.xrs", dest_path=result_folder)

#
# Loads a dict file and extracts spectrum values and calculates the spectrum.
# Loads an already calculated spectrum.
load_folder = Path(__file__).parent / "Test_tub3.dict"
wrapper.load_tube(
    path_dict=load_folder, spec_name="Test_tub3.xrs", path_dest=result_folder
)
wrapper.load_spectra((result_folder / "Tube3_spc.xrs"))

# Runs the current project, starts the simulation.
# Saves the image to a path with a given name.
wrapper.run_project()
wrapper.save_img_project(result_folder, "Pic2.tif")

# Deletes a part with the given ID.
# Resets artist.
wrapper.delete_part(ID=1)
wrapper.clear_project()

# closes aRTist.
# a.close_artist()
```

Usage of the part

There is also an object-oriented way to control aRTist. The following code demonstrates the basic usage.

```
import artistlib as a

path = "$Xray(LibraryDir)/ExampleProjects/Helix-CT.aRTist"
path1 = "$Xray(LibraryDir)/ExampleParts/Fun/Dragon.stl"

result_folder_pic = "$env(HOME) Pictures/"

# pro handles the project and creates the parts from aRTist
pro = a.Project()
pillow, patient, table, tomograph = pro.in_parts(path)

# changes the values in python
patient.x = -12
table.x = -15
pillow.x = 20
```

```
tomograph.x = -4

patient.angleX = 90
patient.angleZ = 90

# Sends the changes to aRTist.
pro.update_project()

Dragon = pro.in_parts(path1)[0]
Dragon.z = 30
pro.update_project()

Dragon.x = 16

# Updates only the single part.
Dragon.update_part()

# Deletes the part.
pillow.delete_part()

patient.y = 20

Dragon.z = 10

pro.update_project()

# Changes the detector.
det = pro.detector()
det.x = 50
det.angleX = 10
det.pixel = 5000
det.size = 250
det.update_detector()

# Changes the source.
src = pro.source()
src.angleX = 5
src.x = 19
src.V = 100
src.Tube = "Test_Name"
src.angleZ = 10
src.res = 20
src.update_source()
src.cal_spectra()

# Runs the simulation and saves the picture.
pro.A.run_project()
pro.A.save_img_project(result_folder_pic, "Pic_oop.tiff")
```

Sub-modules

- artistlib.junction
- artistlib.material
- artistlib.wrapper

Functions

Function `start_artist`

```
def start_artist(
    path_specific_version=None,
    start_UI: bool = True,
    port=3658,
    multiple_starts=False
) -> None
```

Starts aRTist from a Python script. The function searches for the default installation directory or prompts the user to start aRTist manually. The installation path is stored in a JSON file. After the first execution, this stored path is used automatically. Alternatively, a specific installation path can be provided. Set `start_UI=True` to display the aRTist user interface.

Function `close_artist`

```
def close_artist(
    port=3658
) -> None
```

Closes aRTist and resets the junction.

Function `hide_ui`

```
def hide_ui(
    name: str = 'aRTist.exe'
) -> None
```

Hides the aRTist user interface.

Function `show_ui`

```
def show_ui(
    name: str = 'aRTist.exe'
) -> None
```

Shows the aRTist user interface.

Classes

Class `Junction`

```
class Junction(
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

Provides remote control functionality for the aRTist simulator.

Methods

Method `connect`

```
def connect(
    self
)
```

Method `send`

```
def send(
    self,
    command,
    msgType='RESULT'
)
```

Method `listen`

```
def listen(
    self,
    command_no=1,
    msgType='RESULT'
)
```

Method `pick`

```
def pick(
    self,
    answer,
    res='RESULT'
)
```

Method `get_answer`

```
def get_answer(
    self,
    key
)
```

Method `disconnect`

```
def disconnect(
    self
)
```

Method `get_image`

```
def get_image(
    self,
    imageName: str
) -> tuple[PIL.Image.Image, numpy.ndarray]
```

Method `save_disk`

```
def save_disk(
    self,
    im: numpy.ndarray,
    imageName: str
) -> None
```

Method `save_disk_imgObject`

```
def save_disk_imgObject(
    self,
    Img: PIL.Image.Image,
    imageName: str
) -> None
```

Method `receive_file`

```
def receive_file(
    self,
    fileName
)
```

Method `send_file`

```
def send_file(
    self,
    fileName
)
```

Class `ArtistSend`

```
class ArtistSend(
    batch: bool = None,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

This class validates user input and sends the resulting Tcl commands to aRTist.

Descendants

- artistlib.wrapper.artist_detector.ArtistDetector
- artistlib.wrapper.artist_project.ArtistProject
- artistlib.wrapper.artist_setup.ArtistSetup
- artistlib.wrapper.artist_source.ArtistSource
- artistlib.wrapper.artistwrapper_all.ArtistAll
- artistlib.wrapper.material2.MaterialService
- artistlib.wrapper.modules.CT

Static methods

`Method set_global_batch`

```
def set_global_batch(
    value: bool
)
```

Enables batching mode when set to True. Function calls are stored and sent together. **Note:** Batching can improve performance, but some functions may behave differently. Use the apply() function to send the accumulated commands.

Methods

Method `apply`

```
def apply(
    self
) -> str
```

Sends the batched command list to aRTist in chunks.

Method `send`

```
def send(
    self,
    cmd: str
) -> str
```

Sends Tcl commands directly to aRTist or stores them in the batch queue if batching is enabled.

Method `load_file`

```
def load_file(
    self,
    path: str
) -> str
```

Loads a file into aRTist from the specified path.

Class `ArtistProject`

```
class ArtistProject(
    batch: bool = None,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

This class manages a project, including loading, running, and saving project images.

Ancestors (in MRO)

- artistlib.wrapper.artist_send.ArtistSend

Methods

Method `load_save_project`

```
def load_save_project(
    self,
    project_path: str,
    save_img_to: str,
    img_name: str
) -> str
```

Loads a project from a specified path, runs aRTist, saves the resulting image to the specified location, and returns the aRTist response.

Method `load_project`

```
def load_project(
    self,
    path_to_project: str
) -> str
```

Loads a project into aRTist.

Method `run_project`

```
def run_project(
    self
) -> str
```

Runs aRTist using the current project state.

Method `save_img_project`

```
def save_img_project(
    self,
    path_save_img: str,
    img_name: str
) -> str
```

Saves the calculated image to a specified directory.

Method `clear_project`

```
def clear_project(
    self
) -> str
```

Clears the current aRTist workspace.

Method `save_project`

```
def save_project(
    self
) -> str
```

Saves the current project.

Class `ArtistSetup`

```
class ArtistSetup(
    batch: bool = None,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

This class provides functions for controlling the aRTist setup, including moving, rotating, and querying the positions of parts, the detector, and the source.

Ancestors (in MRO)

- artistlib.wrapper.artist_send.ArtistSend

Methods

Method `get_coordinates`

```
def get_coordinates(
    self,
    cmd: str,
    ID: int
) -> tuple[float | None, float | None, float | None]
```

Method `delete_part`

```
def delete_part(
    self,
    ID: int
) -> str
```

Deletes a part with the specified ID.

Method `get_part_global`

```
def get_part_global(
    self,
    ID: int
) -> tuple[float, float, float]
```

Returns the global position of a part. **Note:** Part IDs are integer values starting at 1. The first created part has ID 1, the second has ID 2, and so on.

Method `part_position_global`

```
def part_position_global(
    self,
    ID: int,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Sets the global position of a part. **Note:** Coordinates that are not provided remain unchanged.

Method `get_part_ref`

```
def get_part_ref(
    self,
    ID: int
) -> tuple[float, float, float]
```

Returns the reference position of a part.

Method `part_position_ref`

```
def part_position_ref(
    self,
    ID: int,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Sets the reference position of a part. **Note:** Coordinates that are not provided remain unchanged.

Method `group_transform_rotate`

```
def group_transform_rotate(
    self,
    IDs: list,
    alpha: float,
    x: float,
    y: float,
    z: float,
    Cx: float,
    Cy: float,
    Cz: float
) -> str
```

Rotates a group of parts specified by a list of IDs.

Method `group_transform_translation`

```
def group_transform_translation(
    self,
    IDs: list,
    x: float,
    y: float,
    z: float
) -> str
```

Moves a group of parts specified by a list of IDs.

Method `get_part_orientation`

```
def get_part_orientation(
    self,
    ID: int
) -> tuple[float, float, float]
```

Returns the orientation of a part.

Method `part_orientation`

```
def part_orientation(
    self,
    ID: int,
    angleX: float = None,
    angleY: float = None,
    angleZ: float = None
) -> str
```

Sets the orientation of a part. **Note:** Values that are not provided remain unchanged.

Method `part_rotate`

```
def part_rotate(
    self,
    ID: int,
    system: str,
    alpha: float,
    axisX: float,
    axisY: float,
    axisZ: float
) -> str
```

Rotates a part around the specified x, y, or z axis by the angle alpha. The coordinate system must be specified as either `"world"` or `"object"`

Method `part_rotate_center`

```
def part_rotate_center(
    self,
    ID: int,
    alpha: float,
    x: float,
    y: float,
    z: float,
    Cx: float,
    Cy: float,
    Cz: float
) -> str
```

Rotates a part around a specified center point using the given rotation axis and angle.

Method `part_material`

```
def part_material(
    self,
    ID: int,
    Material: str
) -> str
```

Assigns a material to a part. **Note:** The material must already exist in aRTist.

Method `get_part_material`

```
def get_part_material(
    self,
    ID: int
) -> str
```

Returns the material of the part with the specified ID.

Method `get_detector`

```
def get_detector(
    self
) -> tuple[float, float, float]
```

Returns the current detector position as x, y, z coordinates.

Method `get_source`

```
def get_source(
    self
) -> tuple[float, float, float]
```

Returns the current source position as x, y, z coordinates.

Method `source_position`

```
def source_position(
    self,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Sets the source position.

Method `detector_position`

```
def detector_position(
    self,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Sets the detector position.

Method `get_source_orientation`

```
def get_source_orientation(
    self
) -> tuple[float, float, float]
```

Returns the source orientation.

Method `source_orientation`

```
def source_orientation(
    self,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Sets the source orientation.

Method `get_detector_orientation`

```
def get_detector_orientation(
    self
) -> tuple[float, float, float]
```

Returns the detector orientation.

Method `detector_orientation`

```
def detector_orientation(
    self,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Sets the detector orientation.

Method `get_size`

```
def get_size(
    self,
    ID: int
) -> tuple[float, float, float]
```

Gets the size of the part with the specified ID.

Method `set_size`

```
def set_size(
    self,
    ID: int,
    x: float,
    y: float,
    z: float
) -> str
```

Sets the size of the part with the specified ID.

Method `set_visibility`

```
def set_visibility(
    self,
    ID: int,
    visible: bool = True
) -> str
```

Sets the visibility of a part.

Class `ArtistDetector`

```
class ArtistDetector(
    batch: bool = None,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

This class validates user input and sends the resulting Tcl commands to aRTist.

Ancestors (in MRO)

- artistlib.wrapper.artist_send.ArtistSend

Methods

Method `detector_geo`

```
def detector_geo(
    self,
    size: float = None,
    pixel: float = None,
    res: float = None,
    sizeX: float = None,
    sizeY: float = None,
    pixelX: float = None,
    pixelY: float = None,
    resX: float = None,
    resY: float = None
) -> str
```

Changes the detector geometry by modifying either:

- Pixel count and detector size
- Pixel count and resolution
- Detector size and resolution

The remaining parameter is calculated automatically.

You can provide separate X and Y values for pixel count, detector size, and resolution.

Note:

- If too many parameters are provided, the resolution is recalculated automatically.
- If `pixel=100`, `pixelX=50`, and `pixelY=70` are provided, the function uses `pixelX=50` and `pixelY=70`.

Method `get_detector_pixel_count`

```
def get_detector_pixel_count(
    self
) -> tuple[float, float]
```

Provides the pixel count of the detector. In x and y values.

Method `get_detector_res`

```
def get_detector_res(
    self
) -> tuple[float, float]
```

Provides the resolution of the detector. In x and y values.

Method `get_detector_size`

```
def get_detector_size(
    self
) -> tuple[float, float]
```

Provides the size of the detector. In x and y values.

Method `set_Flat_Field_Correction`

```
def set_Flat_Field_Correction(
    self,
    apply_FFC: bool = False
) -> str
```

Turn the Flat Field Correction of the detector on or off.

Class `ArtistSource`

```
class ArtistSource(
    batch: bool = None,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

This class provides functions for modifying the source.

Ancestors (in MRO)

- artistlib.wrapper.artist_send.ArtistSend

Class variables

Variable `List_content` The type of the None singleton.

Variable `STRING_FIELDS` The type of the None singleton.

Methods

Method `load_spectra`

```
def load_spectra(
    self,
    path: str
) -> str
```

Loads a spectrum from an .xrs file into aRTist.

Method `set_exposure`

```
def set_exposure(
    self,
    exposure: float
) -> str
```

Sets the exposure value in aRTist.

Method `get_exposure`

```
def get_exposure(
    self
)
```

Method `set_spectra`

```
def set_spectra(
    self,
    Tube: str = None,
    V: float = None,
    TMaterial: str = None,
    TAngle: float = None,
    TThick: float = None,
    FMaterial: str = None,
    FThick: float = None,
    F2Material: str = None,
    F2Thick: float = None,
    F3Material: str = None,
    F3Thick: float = None,
    WMaterial: str = None,
    WThick: float = None,
    Angle: float = None,
    Transmission: float = None,
    res: float = None
) -> str
```

Sets individual spectrum parameters.

Method `cal_spectra`

```
def cal_spectra(
    self,
    spec_name: str = None,
    dest_path: str = None,
    reduce_number_bins=False
)
```

Calculates a spectrum and optionally saves it.

Note: * If no output path is specified, the spectrum is calculated but not saved. * Spectrum calculation may take longer for high-voltage configurations. * Set reduce_number_bins to True to reduce the number of bins in the spectra to a maximum of 128 bins.

Method `load_tube`

```
def load_tube(
    self,
    path_dict: str,
    spec_name: str,
    path_dest: str
) -> None
```

Loads a .dict file, applies the spectrum settings, and calculates the resulting spectrum.

Method `get_spectra`

```
def get_spectra(
    self
)
```

Gets the current spectrum and returns it as a dict.

Method `get_spectra_one_value`

```
def get_spectra_one_value(
    self,
    spectrum_parameter
)
```

Returns only the specified spectrum parameter. **Note**: * Possible values are: 'Name', 'Voltage', 'TargetMaterial', 'TargetAngle', 'TargetThickness', 'FilterMaterial', 'FilterThickness', 'Filter2Material', 'Filter2Thickness', 'Filter3Material', 'Filter3Thickness', 'WindowMaterial', 'WindowThickness', 'AngleIn', 'Transmission', 'Resolution'.

Method `set_source_type`

```
def set_source_type(
    self,
    source_type: str = 'Mono'
)
```

Sets the type of the source. **Note** * source_type has to be either "Mono" or "General".

Class `MaterialService`

```
class MaterialService(
    batch: bool = None,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

This class validates user input and sends the resulting Tcl commands to aRTist.

Ancestors (in MRO)

- artistlib.wrapper.artist_send.ArtistSend

Methods

Method `add_material`

```
def add_material(
    self,
    m_name: str = 'dummy',
    density: float = 0,
    composition: str = 'Al',
    comment: str = '(by artistlib)'
)
```

Method `delete_material`

```
def delete_material(
    self,
    m_name: str
)
```

Method `names_material`

```
def names_material(
    self
)
```

Method `pick_material`

```
def pick_material(
    self,
    mname
)
```

Retrieves a material definition from aRTist.

Class `Material`

```
class Material(
    name='dummy'
)
```

Methods

Method `add`

```
def add(
    self
)
```

Method `delete`

```
def delete(
    self
)
```

Method `pick`

```
def pick(
    self,
    name
)
```

Class `ArtistAll`

```
class ArtistAll(
    path_specific_version=None,
    start_UI: bool = True,
    batch: bool = None,
    multiple_starts=False,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

A wrapper class that provides access to all available classes and offers autocomplete support.

Ancestors (in MRO)

- artistlib.wrapper.artist_send.ArtistSend

Methods

Method `load_save_project`

```
def load_save_project(
    self,
    project_path: str,
    save_img_to: str,
    img_name: str
) -> str
```

Method `load_project`

```
def load_project(
    self,
    path_to_project: str
) -> str
```

Method `run_project`

```
def run_project(
    self
) -> str
```

Method `save_img_project`

```
def save_img_project(
    self,
    path_save_img: str,
    img_name: str
) -> str
```

Method `get_part_global`

```
def get_part_global(
    self,
    ID: int
) -> str
```

Method `part_position_global`

```
def part_position_global(
    self,
    ID: int,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Method `get_part_ref`

```
def get_part_ref(
    self,
    ID: int
) -> tuple[float, float, float]
```

Method `part_position_ref`

```
def part_position_ref(
    self,
    ID: int,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Method `get_part_orientation`

```
def get_part_orientation(
    self,
    ID: int
) -> tuple[float, float, float]
```

Method `part_orientation`

```
def part_orientation(
    self,
    ID: int,
    angleX: float = None,
    angleY: float = None,
    angleZ: float = None
) -> str
```

Method `part_rotate`

```
def part_rotate(
    self,
    ID: int,
    system: str,
    alpha: float,
    axisX: float,
    axisY: float,
    axisZ: float
) -> str
```

Method `part_rotate_center`

```
def part_rotate_center(
    self,
    ID: int,
    alpha: float,
    x: float,
    y: float,
    z: float,
    Cx: float,
    Cy: float,
    Cz: float
) -> str
```

Method `part_material`

```
def part_material(
    self,
    ID: int,
    Material: str
) -> str
```

Method `get_detector`

```
def get_detector(
    self
) -> tuple[float, float, float]
```

Method `get_source`

```
def get_source(
    self
) -> tuple[float, float, float]
```

Method `source_position`

```
def source_position(
    self,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Method `detector_position`

```
def detector_position(
    self,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Method `get_source_orientation`

```
def get_source_orientation(
    self
) -> tuple[float, float, float]
```

Method `source_orientation`

```
def source_orientation(
    self,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Method `get_detector_orientation`

```
def get_detector_orientation(
    self
)
```

Method `detector_orientation`

```
def detector_orientation(
    self,
    x: float = None,
    y: float = None,
    z: float = None
)
```

Method `detector_geo`

```
def detector_geo(
    self,
    size: float = None,
    pixel: float = None,
    res: float = None,
    sizeX: float = None,
    sizeY: float = None,
    pixelX: float = None,
    pixelY: float = None,
    resX: float = None,
    resY: float = None
)
```

Method `load_spectra`

```
def load_spectra(
    self,
    path: str
)
```

Method `set_spectra`

```
def set_spectra(
    self,
    Tube: str = None,
    V: float = None,
    TMaterial: str = None,
    TAngle: float = None,
    TThick: float = None,
    FMaterial: str = None,
    FThick: float = None,
    F2Material: str = None,
    F2Thick: float = None,
    F3Material: str = None,
    F3Thick: float = None,
    WMaterial: str = None,
    WThick: float = None,
    Angle: float = None,
    Transmission: float = None,
    res: float = None
)
```

Method `cal_spectra`

```
def cal_spectra(
    self,
    spec_name: str = None,
    dest_path: str = None,
    reduce_number_bins=False
)
```

Method `load_tube`

```
def load_tube(
    self,
    path_dict: str,
    spec_name: str,
    path_dest: str
```

```
)
```

Method `set_exposure`

```
def set_exposure(
    self,
    exposure: float
)
```

Method `delete_part`

```
def delete_part(
    self,
    ID: int
)
```

Method `clear_project`

```
def clear_project(
    self
)
```

Method `save_project`

```
def save_project(
    self
)
```

Method `get_part_material`

```
def get_part_material(
    self,
    ID: int
)
```

Method `get_detector_pixel_count`

```
def get_detector_pixel_count(
    self
)
```

Method `get_detector_res`

```
def get_detector_res(
    self
)
```

Method `get_detector_size`

```
def get_detector_size(
    self
)
```

Method `get_size`

```
def get_size(
    self,
    ID: int
)
```

Method `set_size`

```
def set_size(
    self,
    ID: int,
    x: float,
    y: float,
    z: float
)
```

Method `get_spectra`

```
def get_spectra(
    self
)
```

Method `get_spectra_one_value`

```
def get_spectra_one_value(
    self,
    spectrum_parameter: str
)
```

Method `get_exposure`

```
def get_exposure(
    self
)
```

Method `set_source_type`

```
def set_source_type(
    self,
    source_type='Mono'
)
```

Method `set_Flat_Field_Correction`

```
def set_Flat_Field_Correction(
    self,
    apply_FFC=False
)
```

Method `set_visibility`

```
def set_visibility(
    self,
    ID: int,
    visible: bool = True
)
```

Class `CT`

```
class CT(
    batch: bool = None,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

Provides access to the CTScan module through Python

Ancestors (in MRO)

- artistlib.wrapper.artist_send.ArtistSend

Methods

Method `run_CT`

```
def run_CT(
    self
) -> str
```

Runs a CT scan.

Method `set_CT`

```
def set_CT(
    self,
    Angle: float = None,
    Steps: float = None,
    path: str = None,
    File_Name: str = None,
    File_Type: str = None,
    run_Feldkamp: bool = None,
    PixelType: str = None,
    Direction: str = None,
    Scattermode: str = None,
    ScatterIntervall: float = None
) -> str
```

Configures all CTScan parameters and validates the provided arguments. * File_Type must be either “{BAM CT}” or “TIFFs”. * PixelType must be either “float” or “16bit”. * Direction must be either “clockwise” or “counterclockwise”. * Scattermode must be either “{force McRay}” or “{as configured}”. **Note:** The Feldkamp reconstruction method is only available when `File_Type = "{BAM CT}"`.

Class `Project`

```
class Project
```

This class manages project parts by creating corresponding Python objects and synchronizing them with the parts in aRTist.

Methods

Method `in_parts`

```
def in_parts(
    self,
    path: str
) -> list[artistlib.wrapper.part.Part]
```

Creates Python objects from the parts currently available in aRTist.

Method `clear_project`

```
def clear_project(
    self
) -> list[artistlib.wrapper.part.Part]
```

Clears the virtual scene.

Method `update_project`

```
def update_project(
    self
) -> None
```

This function updates the state of all parts in this project.

Method `source`

```
def source(
    self
) -> artistlib.wrapper.part.Source
```

Method `detector`

```
def detector(
    self
) -> artistlib.wrapper.part.Detector
```

Class `Part`

```
class Part(
    project_obj,
    ID,
    token
)
```

This class creates a Python object based on the configuration in Artist.

Methods

Method `update_part`

```
def update_part(
    self
) -> str
```

Applies modifications to Artist.

Method `delete_part`

```
def delete_part(
    self
) -> str
```

Removes the specified part from Artist.

Class `Detector`

```
class Detector(
    project_obj
)
```

Provides an object-oriented interface for the aRTist detector.

Methods

Method `update_detector`

```
def update_detector(
    self
) -> str
```

Applies modifications to the detector configuration in Artist.

Class `Source`

```
class Source(
    project_obj
)
```

Provides an object-oriented interface for the aRTist source.

Methods

Method `update_source`

```
def update_source(
    self
) -> str
```

Applies modifications to the source configuration in Artist.

Method `cal_spectra`

```
def cal_spectra(
    self,
    spec_name: str = None,
    dest_path: str = None
) -> str
```

Finally, recalculates the spectrum after all modifications have been applied.

Module `artistlib.junction`

Creates the remote control.

Classes

Class `Junction`

```
class Junction(
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

Provides remote control functionality for the aRTist simulator.

Methods

Method `connect`

```
def connect(
    self
)
```

Method `send`

```
def send(
    self,
    command,
    msgType='RESULT'
)
```

Method `listen`

```
def listen(
    self,
    command_no=1,
    msgType='RESULT'
)
```

Method `pick`

```
def pick(
    self,
    answer,
    res='RESULT'
)
```

Method `get_answer`

```
def get_answer(
    self,
    key
)
```

Method `disconnect`

```
def disconnect(
    self
)
```

Method `get_image`

```
def get_image(
    self,
    imageName: str
) -> tuple[PIL.Image.Image, numpy.ndarray]
```

Method `save_disk`

```
def save_disk(
    self,
    im: numpy.ndarray,
    imageName: str
) -> None
```

Method `save_disk_imgObject`

```
def save_disk_imgObject(
    self,
    Img: PIL.Image.Image,
    imageName: str
) -> None
```

Method `receive_file`

```
def receive_file(
    self,
    fileName
)
```

Method `send_file`

```
def send_file(
    self,
    fileName
)
```

Module `artistlib.material`

Define and handle materials

Functions

Function `brace_to_quotes`

```
def brace_to_quotes(
    text
)
```

Helper functions for handling Tcl dictionaries.

Classes

Class `Material`

```
class Material(
    name='dummy'
)
```

Defines a material

Instance variables

Variable `name` Returns the material name.

Variable `definition` Returns a material entry in the format required by the aRTist material list.

Methods

Method `add`

```
def add(
    self,
    junction
)
```

Adds or updates the material in aRTist.

Method `delete`

```
def delete(
    self,
    junction
)
```

Deletes the material from aRTist.

Method `names`

```
def names(
    self,
    junction
)
```

Returns a list of all materials currently available in aRTist.

Method `pick`

```
def pick(
    self,
    junction,
    name
)
```

Retrieves a material definition from aRTist.

Module `artistlib.wrapper`

Wrapper for simple use of aRTist.

Sub-modules

- artistlib.wrapper.artist_detector
- artistlib.wrapper.artist_project
- artistlib.wrapper.artist_send
- artistlib.wrapper.artist_setup
- artistlib.wrapper.artist_source
- artistlib.wrapper.artistwrapper_all
- artistlib.wrapper.material2
- artistlib.wrapper.modules
- artistlib.wrapper.part
- artistlib.wrapper.start

Module `artistlib.wrapper.artist_detector`

Classes

Class `ArtistDetector`

```
class ArtistDetector(
    batch: bool = None,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

This class validates user input and sends the resulting Tcl commands to aRTist.

Ancestors (in MRO)

- artistlib.wrapper.artist_send.ArtistSend

Methods

Method `detector_geo`

```
def detector_geo(
    self,
    size: float = None,
    pixel: float = None,
    res: float = None,
    sizeX: float = None,
    sizeY: float = None,
    pixelX: float = None,
    pixelY: float = None,
    resX: float = None,
    resY: float = None
) -> str
```

Changes the detector geometry by modifying either:

- Pixel count and detector size
- Pixel count and resolution
- Detector size and resolution

The remaining parameter is calculated automatically.

You can provide separate X and Y values for pixel count, detector size, and resolution.

Note:

- If too many parameters are provided, the resolution is recalculated automatically.
- If `pixel=100`, `pixelX=50`, and `pixelY=70` are provided, the function uses `pixelX=50` and `pixelY=70`.

Method `get_detector_pixel_count`

```
def get_detector_pixel_count(
    self
) -> tuple[float, float]
```

Provides the pixel count of the detector. In x and y values.

Method `get_detector_res`

```
def get_detector_res(
    self
) -> tuple[float, float]
```

Provides the resolution of the detector. In x and y values.

Method `get_detector_size`

```
def get_detector_size(
    self
) -> tuple[float, float]
```

Provides the size of the detector. In x and y values.

Method `set_Flat_Field_Correction`

```
def set_Flat_Field_Correction(
    self,
    apply_FFC: bool = False
) -> str
```

Turn the Flat Field Correction of the detector on or off.

Module `artistlib.wrapper.artist_project`

Classes

Class `ArtistProject`

```
class ArtistProject(
    batch: bool = None,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

This class manages a project, including loading, running, and saving project images.

Ancestors (in MRO)

- artistlib.wrapper.artist_send.ArtistSend

Methods

Method `load_save_project`

```
def load_save_project(
    self,
    project_path: str,
    save_img_to: str,
    img_name: str
) -> str
```

Loads a project from a specified path, runs aRTist, saves the resulting image to the specified location, and returns the aRTist response.

Method `load_project`

```
def load_project(
    self,
    path_to_project: str
) -> str
```

Loads a project into aRTist.

Method `run_project`

```
def run_project(
    self
) -> str
```

Runs aRTist using the current project state.

Method `save_img_project`

```
def save_img_project(
    self,
    path_save_img: str,
    img_name: str
) -> str
```

Saves the calculated image to a specified directory.

Method `clear_project`

```
def clear_project(
    self
) -> str
```

Clears the current aRTist workspace.

Method `save_project`

```
def save_project(
    self
) -> str
```

Saves the current project.

Module `artistlib.wrapper.artist_send`

Classes

Class `ArtistSend`

```
class ArtistSend(
    batch: bool = None,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

This class validates user input and sends the resulting Tcl commands to aRTist.

Descendants

- artistlib.wrapper.artist_detector.ArtistDetector
- artistlib.wrapper.artist_project.ArtistProject
- artistlib.wrapper.artist_setup.ArtistSetup
- artistlib.wrapper.artist_source.ArtistSource
- artistlib.wrapper.artistwrapper_all.ArtistAll
- artistlib.wrapper.material2.MaterialService
- artistlib.wrapper.modules.CT

Static methods

`Method set_global_batch`

```
def set_global_batch(
    value: bool
)
```

Enables batching mode when set to True. Function calls are stored and sent together. **Note:** Batching can improve performance, but some functions may behave differently. Use the apply() function to send the accumulated commands.

Methods

Method `apply`

```
def apply(
    self
) -> str
```

Sends the batched command list to aRTist in chunks.

Method `send`

```
def send(
    self,
    cmd: str
) -> str
```

Sends Tcl commands directly to aRTist or stores them in the batch queue if batching is enabled.

Method `load_file`

```
def load_file(
    self,
    path: str
) -> str
```

Loads a file into aRTist from the specified path.

Module `artistlib.wrapper.artist_setup`

Classes

Class `ArtistSetup`

```
class ArtistSetup(
    batch: bool = None,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

This class provides functions for controlling the aRTist setup, including moving, rotating, and querying the positions of parts, the detector, and the source.

Ancestors (in MRO)

- artistlib.wrapper.artist_send.ArtistSend

Methods

Method `get_coordinates`

```
def get_coordinates(
    self,
    cmd: str,
    ID: int
) -> tuple[float | None, float | None, float | None]
```

Method `delete_part`

```
def delete_part(
    self,
    ID: int
) -> str
```

Deletes a part with the specified ID.

Method `get_part_global`

```
def get_part_global(
    self,
    ID: int
) -> tuple[float, float, float]
```

Returns the global position of a part. **Note:** Part IDs are integer values starting at 1. The first created part has ID 1, the second has ID 2, and so on.

Method `part_position_global`

```
def part_position_global(
    self,
    ID: int,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Sets the global position of a part. **Note:** Coordinates that are not provided remain unchanged.

Method `get_part_ref`

```
def get_part_ref(
    self,
    ID: int
) -> tuple[float, float, float]
```

Returns the reference position of a part.

Method `part_position_ref`

```
def part_position_ref(
    self,
    ID: int,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Sets the reference position of a part. **Note:** Coordinates that are not provided remain unchanged.

Method `group_transform_rotate`

```
def group_transform_rotate(
    self,
    IDs: list,
    alpha: float,
    x: float,
    y: float,
    z: float,
    Cx: float,
    Cy: float,
    Cz: float
) -> str
```

Rotates a group of parts specified by a list of IDs.

Method `group_transform_translation`

```
def group_transform_translation(
    self,
    IDs: list,
    x: float,
    y: float,
    z: float
) -> str
```

Moves a group of parts specified by a list of IDs.

Method `get_part_orientation`

```
def get_part_orientation(
    self,
    ID: int
) -> tuple[float, float, float]
```

Returns the orientation of a part.

Method `part_orientation`

```
def part_orientation(
    self,
    ID: int,
    angleX: float = None,
    angleY: float = None,
    angleZ: float = None
) -> str
```

Sets the orientation of a part. **Note:** Values that are not provided remain unchanged.

Method `part_rotate`

```
def part_rotate(
    self,
    ID: int,
    system: str,
    alpha: float,
    axisX: float,
    axisY: float,
    axisZ: float
) -> str
```

Rotates a part around the specified x, y, or z axis by the angle alpha. The coordinate system must be specified as either `"world"` or `"object"`

Method `part_rotate_center`

```
def part_rotate_center(
    self,
    ID: int,
    alpha: float,
    x: float,
    y: float,
    z: float,
    Cx: float,
    Cy: float,
    Cz: float
) -> str
```

Rotates a part around a specified center point using the given rotation axis and angle.

Method `part_material`

```
def part_material(
    self,
    ID: int,
    Material: str
) -> str
```

Assigns a material to a part. **Note:** The material must already exist in aRTist.

Method `get_part_material`

```
def get_part_material(
    self,
    ID: int
) -> str
```

Returns the material of the part with the specified ID.

Method `get_detector`

```
def get_detector(
    self
) -> tuple[float, float, float]
```

Returns the current detector position as x, y, z coordinates.

Method `get_source`

```
def get_source(
    self
) -> tuple[float, float, float]
```

Returns the current source position as x, y, z coordinates.

Method `source_position`

```
def source_position(
    self,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Sets the source position.

Method `detector_position`

```
def detector_position(
    self,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Sets the detector position.

Method `get_source_orientation`

```
def get_source_orientation(
    self
) -> tuple[float, float, float]
```

Returns the source orientation.

Method `source_orientation`

```
def source_orientation(
    self,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Sets the source orientation.

Method `get_detector_orientation`

```
def get_detector_orientation(
    self
) -> tuple[float, float, float]
```

Returns the detector orientation.

Method `detector_orientation`

```
def detector_orientation(
    self,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Sets the detector orientation.

Method `get_size`

```
def get_size(
    self,
    ID: int
) -> tuple[float, float, float]
```

Gets the size of the part with the specified ID.

Method `set_size`

```
def set_size(
    self,
    ID: int,
    x: float,
    y: float,
    z: float
) -> str
```

Sets the size of the part with the specified ID.

Method `set_visibility`

```
def set_visibility(
    self,
    ID: int,
    visible: bool = True
) -> str
```

Sets the visibility of a part.

Module `artistlib.wrapper.artist_source`

Classes

Class `ArtistSource`

```
class ArtistSource(
    batch: bool = None,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

This class provides functions for modifying the source.

Ancestors (in MRO)

- artistlib.wrapper.artist_send.ArtistSend

Class variables

Variable `List_content` The type of the None singleton.

Variable `STRING_FIELDS` The type of the None singleton.

Methods

Method `load_spectra`

```
def load_spectra(
    self,
    path: str
) -> str
```

Loads a spectrum from an .xrs file into aRTist.

Method `set_exposure`

```
def set_exposure(
    self,
    exposure: float
) -> str
```

Sets the exposure value in aRTist.

Method `get_exposure`

```
def get_exposure(
    self
)
```

Method `set_spectra`

```
def set_spectra(
    self,
    Tube: str = None,
    V: float = None,
    TMaterial: str = None,
    TAngle: float = None,
    TThick: float = None,
```

```
    FMaterial: str = None,
    FThick: float = None,
    F2Material: str = None,
    F2Thick: float = None,
    F3Material: str = None,
    F3Thick: float = None,
    WMaterial: str = None,
    WThick: float = None,
    Angle: float = None,
    Transmission: float = None,
    res: float = None
) -> str
```

Sets individual spectrum parameters.

Method `cal_spectra`

```
def cal_spectra(
    self,
    spec_name: str = None,
    dest_path: str = None,
    reduce_number_bins=False
)
```

Calculates a spectrum and optionally saves it.

Note: * If no output path is specified, the spectrum is calculated but not saved. * Spectrum calculation may take longer for high-voltage configurations. * Set reduce_number_bins to True to reduce the number of bins in the spectra to a maximum of 128 bins.

Method `load_tube`

```
def load_tube(
    self,
    path_dict: str,
    spec_name: str,
    path_dest: str
) -> None
```

Loads a .dict file, applies the spectrum settings, and calculates the resulting spectrum.

Method `get_spectra`

```
def get_spectra(
    self
)
```

Gets the current spectrum and returns it as a dict.

Method `get_spectra_one_value`

```
def get_spectra_one_value(
    self,
    spectrum_parameter
)
```

Returns only the specified spectrum parameter. **Note**: * Possible values are: ‘Name’, ‘Voltage’, ‘TargetMaterial’, ‘TargetAngle’, ‘TargetThickness’, ‘FilterMaterial’, ‘FilterThickness’, ‘Filter2Material’, ‘Filter2Thickness’, ‘Filter3Material’, ‘Filter3Thickness’, ‘WindowMaterial’, ‘WindowThickness’, ‘AngleIn’, ‘Transmission’, ‘Resolution’.

Method `set_source_type`

```
def set_source_type(
    self,
    source_type: str = 'Mono'
)
```

Sets the type of the source. **Note** * source_type has to be either “Mono” or “General”.

Module `artistlib.wrapper.artistwrapper_all`

Classes

Class `ArtistAll`

```
class ArtistAll(
    path_specific_version=None,
    start_UI: bool = True,
    batch: bool = None,
    multiple_starts=False,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

A wrapper class that provides access to all available classes and offers autocomplete support.

Ancestors (in MRO)

- artistlib.wrapper.artist_send.ArtistSend

Methods

Method `load_save_project`

```
def load_save_project(
    self,
    project_path: str,
    save_img_to: str,
    img_name: str
) -> str
```

Method `load_project`

```
def load_project(
    self,
    path_to_project: str
) -> str
```

Method `run_project`

```
def run_project(
    self
) -> str
```

Method `save_img_project`

```
def save_img_project(
    self,
    path_save_img: str,
```

```
        img_name: str
    ) -> str
```

Method `get_part_global`

```
    def get_part_global(
        self,
        ID: int
    ) -> str
```

Method `part_position_global`

```
    def part_position_global(
        self,
        ID: int,
        x: float = None,
        y: float = None,
        z: float = None
    ) -> str
```

Method `get_part_ref`

```
    def get_part_ref(
        self,
        ID: int
    ) -> tuple[float, float, float]
```

Method `part_position_ref`

```
    def part_position_ref(
        self,
        ID: int,
        x: float = None,
        y: float = None,
        z: float = None
    ) -> str
```

Method `get_part_orientation`

```
    def get_part_orientation(
        self,
        ID: int
    ) -> tuple[float, float, float]
```

Method `part_orientation`

```
    def part_orientation(
        self,
        ID: int,
        angleX: float = None,
        angleY: float = None,
        angleZ: float = None
    ) -> str
```

Method `part_rotate`

```
    def part_rotate(
        self,
        ID: int,
        system: str,
```

```
        alpha: float,
        axisX: float,
        axisY: float,
        axisZ: float
    ) -> str
```

Method `part_rotate_center`

```
    def part_rotate_center(
        self,
        ID: int,
        alpha: float,
        x: float,
        y: float,
        z: float,
        Cx: float,
        Cy: float,
        Cz: float
    ) -> str
```

Method `part_material`

```
    def part_material(
        self,
        ID: int,
        Material: str
    ) -> str
```

Method `get_detector`

```
    def get_detector(
        self
    ) -> tuple[float, float, float]
```

Method `get_source`

```
    def get_source(
        self
    ) -> tuple[float, float, float]
```

Method `source_position`

```
    def source_position(
        self,
        x: float = None,
        y: float = None,
        z: float = None
    ) -> str
```

Method `detector_position`

```
    def detector_position(
        self,
        x: float = None,
        y: float = None,
        z: float = None
    ) -> str
```

Method `get_source_orientation`

```
def get_source_orientation(
    self
) -> tuple[float, float, float]
```

Method `source_orientation`

```
def source_orientation(
    self,
    x: float = None,
    y: float = None,
    z: float = None
) -> str
```

Method `get_detector_orientation`

```
def get_detector_orientation(
    self
)
```

Method `detector_orientation`

```
def detector_orientation(
    self,
    x: float = None,
    y: float = None,
    z: float = None
)
```

Method `detector_geo`

```
def detector_geo(
    self,
    size: float = None,
    pixel: float = None,
    res: float = None,
    sizeX: float = None,
    sizeY: float = None,
    pixelX: float = None,
    pixelY: float = None,
    resX: float = None,
    resY: float = None
)
```

Method `load_spectra`

```
def load_spectra(
    self,
    path: str
)
```

Method `set_spectra`

```
def set_spectra(
    self,
    Tube: str = None,
    V: float = None,
    TMaterial: str = None,
    TAngle: float = None,
```

```
    TThick: float = None,
    FMaterial: str = None,
    FThick: float = None,
    F2Material: str = None,
    F2Thick: float = None,
    F3Material: str = None,
    F3Thick: float = None,
    WMaterial: str = None,
    WThick: float = None,
    Angle: float = None,
    Transmission: float = None,
    res: float = None
)
```

Method `cal_spectra`

```
def cal_spectra(
    self,
    spec_name: str = None,
    dest_path: str = None,
    reduce_number_bins=False
)
```

Method `load_tube`

```
def load_tube(
    self,
    path_dict: str,
    spec_name: str,
    path_dest: str
)
```

Method `set_exposure`

```
def set_exposure(
    self,
    exposure: float
)
```

Method `delete_part`

```
def delete_part(
    self,
    ID: int
)
```

Method `clear_project`

```
def clear_project(
    self
)
```

Method `save_project`

```
def save_project(
    self
)
```

Method `get_part_material`

```
def get_part_material(
    self,
    ID: int
)
```

Method `get_detector_pixel_count`

```
def get_detector_pixel_count(
    self
)
```

Method `get_detector_res`

```
def get_detector_res(
    self
)
```

Method `get_detector_size`

```
def get_detector_size(
    self
)
```

Method `get_size`

```
def get_size(
    self,
    ID: int
)
```

Method `set_size`

```
def set_size(
    self,
    ID: int,
    x: float,
    y: float,
    z: float
)
```

Method `get_spectra`

```
def get_spectra(
    self
)
```

Method `get_spectra_one_value`

```
def get_spectra_one_value(
    self,
    spectrum_parameter: str
)
```

Method `get_exposure`

```
def get_exposure(
    self
)
```

Method `set_source_type`

```
def set_source_type(
    self,
    source_type='Mono'
)
```

Method `set_Flat_Field_Correction`

```
def set_Flat_Field_Correction(
    self,
    apply_FFC=False
)
```

Method `set_visibility`

```
def set_visibility(
    self,
    ID: int,
    visible: bool = True
)
```

Module `artistlib.wrapper.material2`

Classes

Class `MaterialService`

```
class MaterialService(
    batch: bool = None,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

This class validates user input and sends the resulting Tcl commands to aRTist.

Ancestors (in MRO)

- artistlib.wrapper.artist_send.ArtistSend

Methods

Method `add_material`

```
def add_material(
    self,
    m_name: str = 'dummy',
    density: float = 0,
    composition: str = 'Al',
    comment: str = '(by artistlib)'
)
```

Method `delete_material`

```
def delete_material(
    self,
    m_name: str
)
```

Method `names_material`

```
def names_material(
    self
)
```

Method `pick_material`

```
def pick_material(
    self,
    mname
)
```

Retrieves a material definition from aRTist.

Class `Material`

```
class Material(
    name='dummy'
)
```

Methods

Method `add`

```
def add(
    self
)
```

Method `delete`

```
def delete(
    self
)
```

Method `pick`

```
def pick(
    self,
    name
)
```

Module `artistlib.wrapper.modules`

Class to use CtScan Modul via Python

Classes

Class `CT`

```
class CT(
    batch: bool = None,
    host='localhost',
    port=3658,
    bufferSize=1024,
    timeout=5
)
```

Provides access to the CTScan module through Python

Ancestors (in MRO)

- artistlib.wrapper.artist_send.ArtistSend

Methods

Method `run_CT`

```
def run_CT(
    self
) -> str
```

Runs a CT scan.

Method `set_CT`

```
def set_CT(
    self,
    Angle: float = None,
    Steps: float = None,
    path: str = None,
    File_Name: str = None,
    File_Type: str = None,
    run_Feldkamp: bool = None,
    PixelType: str = None,
    Direction: str = None,
    Scattermode: str = None,
    ScatterIntervall: float = None
) -> str
```

Configures all CTScan parameters and validates the provided arguments. * File_Type must be either “{BAM CT}” or “TIFFs”. * PixelType must be either “float” or “16bit”. * Direction must be either “clockwise” or “counterclockwise”. * Scattermode must be either “{force McRay}” or “{as configured}”. **Note:** The Feldkamp reconstruction method is only available when `File_Type = "{BAM CT}"`.

Module `artistlib.wrapper.part`

Classes

Class `Project`

```
class Project
```

This class manages project parts by creating corresponding Python objects and synchronizing them with the parts in aRTist.

Methods

Method `in_parts`

```
def in_parts(
    self,
    path: str
) -> list[artistlib.wrapper.part.Part]
```

Creates Python objects from the parts currently available in aRTist.

Method `clear_project`

```
def clear_project(
    self
) -> list[artistlib.wrapper.part.Part]
```

Clears the virtual scene.

Method `update_project`

```
def update_project(
    self
) -> None
```

This function updates the state of all parts in this project.

Method `source`

```
def source(
    self
) -> artistlib.wrapper.part.Source
```

Method `detector`

```
def detector(
    self
) -> artistlib.wrapper.part.Detector
```

Class `Part`

```
class Part(
    project_obj,
    ID,
    token
)
```

This class creates a Python object based on the configuration in Artist.

Methods

Method `update_part`

```
def update_part(
    self
) -> str
```

Applies modifications to Artist.

Method `delete_part`

```
def delete_part(
    self
) -> str
```

Removes the specified part from Artist.

Class `Detector`

```
class Detector(
    project_obj
)
```

Provides an object-oriented interface for the aRTist detector.

Methods

Method `update_detector`

```
def update_detector(
    self
) -> str
```

Applies modifications to the detector configuration in Artist.

Class `Source`

```
class Source(
    project_obj
)
```

Provides an object-oriented interface for the aRTist source.

Methods

Method `update_source`

```
def update_source(
    self
) -> str
```

Applies modifications to the source configuration in Artist.

Method `cal_spectra`

```
def cal_spectra(
    self,
    spec_name: str = None,
    dest_path: str = None
) -> str
```

Finally, recalculates the spectrum after all modifications have been applied.

Module `artistlib.wrapper.start`

Starts and closes aRTist. Functions to control the UI.

Sub-modules

- artistlib.wrapper.start.artist_start
- artistlib.wrapper.start.file_utility
- artistlib.wrapper.start.find_installation
- artistlib.wrapper.start.launcher
- artistlib.wrapper.start.process_helper
- artistlib.wrapper.start.ui

Module `artistlib.wrapper.start.artist_start`

Functions

Function `start_artist`

```
def start_artist(
    path_specific_version=None,
    start_UI: bool = True,
    port=3658,
    multiple_starts=False
) -> None
```

Starts aRTist from a Python script. The function searches for the default installation directory or prompts the user to start aRTist manually. The installation path is stored in a JSON file. After the first execution, this stored path is used automatically. Alternatively, a specific installation path can be provided. Set `start_UI=True` to display the aRTist user interface.

Function `close_artist`

```
def close_artist(
    port=3658
) -> None
```

Closes aRTist and resets the junction.

Function `show_ui`

```
def show_ui(
    name: str = 'aRTist.exe'
) -> None
```

Shows the aRTist user interface.

Function `hide_ui`

```
def hide_ui(
    name: str = 'aRTist.exe'
) -> None
```

Hides the aRTist user interface.

Module `artistlib.wrapper.start.file_utility`

Functions

Function `find_file`

```
def find_file(
    name,
    path
)
```

Function `read_json`

```
def read_json(
    file
)
```

Function `write_json`

```
def write_json(
    file,
    data
)
```

Function `create_file`

```
def create_file(
    file,
    cmd
)
```

Function `delete_file`

```
def delete_file(
    file
)
```

Function `check_pid`

```
def check_pid(
    pid
)
```

Function `find_file_del`

```
def find_file_del(
    path,
    start,
    end
)
```

Module `artistlib.wrapper.start.find_installation`

Functions

Function `find_artist_installation`

```
def find_artist_installation(
    jfile,
    path_std_list,
    name='aRTist.exe'
)
```

Function `find_artist_manual`

```
def find_artist_manual(
    name,
    jfile,
    port=3658
)
```

Module `artistlib.wrapper.start.launcher`

Functions

Function `load_artist_json`

```
def load_artist_json(
    jfile
)
```

Function `run_artist`

```
def run_artist(
    cmd_str,
    tcl_file
)
```

Function `find_and_start`

```
def find_and_start(
    path_specific_version,
    jfile,
    path_std_list,
    tcl_file,
    tcl_file_content,
    flag_path,
    name
)
```

Function `launch_artist`

```
def launch_artist(
    path_specific_version=None,
    start_UI=True,
    port=3658,
    multiple_starts=False
)
```

Function `close_artist`

```
def close_artist(
    port=3658
)
```

Module `artistlib.wrapper.start.process_helper`

Functions

Function `is_port_open`

```
def is_port_open(
    host,
    port
)
```

Function `wait_to_start`

```
def wait_to_start(
    flag_path,
    timeout=100
)
```

Function `check_running_win`

```
def check_running_win() -> bool
```

Function `check_running_Linux`

```
def check_running_Linux() -> bool
```

Module `artistlib.wrapper.start.ui`

Functions

Function `get_pid_by_name`

```
def get_pid_by_name(
```

```
    process_name
)
```

Function `get_hwnds_by_pid`

```
def get_hwnds_by_pid(
    pid
)
```

Function `hide_window`

```
def hide_window(
    hwnds
)
```

Function `show_window`

```
def show_window(
    hwnds
)
```

Function `hide_ui`

```
def hide_ui(
    name
)
```

Function `show_ui`

```
def show_ui(
    name
)
```

Generated by *pdoc* 0.11.6 (https://pdoc3.github.io).